

Week 1: Project Scaffolding, Frontend Setup & Vercel CI/CD

Yash

Roll No: 1024030846

27 August 2026

1 Context

Start of the SkillGrid project. The GitHub repository existed but was empty. Today's work was to bring the repository up to the structure required by the UCS503P course template, and to get a first, real, automatically deployed increment of the frontend working – in line with the course's emphasis on rapid, CI/CD-backed iterations.

2 Repository Structure

Cloned the reference course template (`tiet-ucs503/ucs503p-202627odd-template`) to study its layout, then replicated the same structure in the SkillGrid repository:

- `docs/` – MkDocs-based documentation site.
- `journals/` – one folder per team member (this file lives under `journals/1024030846-yash/`).
- `project-proposal/`, `project-report-prototype-stage/`, `project-report-final/` – LaTeX report folders.
- `code/` – source code root.
- `assets/`, `mkdocs.yml`, `.github/workflows/mkdocs.yml` – documentation build and GitHub Pages deployment.

2.1 Issue: Symlinks on Windows

The template links `docs/assets` and `docs/journals` to the root-level `assets/` and `journals/` folders via symlinks, so MkDocs can serve them without duplicating content.

Problem: creating these with `ln -s` on Windows (without Developer Mode/admin privilege) silently created real directories instead of symlinks, which then got fully duplicated into the git index as separate binary files.

Fix: removed the duplicated directories and used git's plumbing commands directly to record proper symlink blob entries (git file mode 120000):

```
printf '../assets' | git hash-object -w --stdin
git update-index --add --cacheinfo 120000,<sha>,docs/assets
```

This checks out as an inert placeholder file on Windows (no symlink support), but resolves as a real symlink on the Ubuntu-based GitHub Actions runner, which is where the MkDocs build actually happens.

3 Frontend Scaffold

Since `code/` was empty, scaffolded a minimal Next.js application (TypeScript, ESLint, Tailwind CSS, App Router) under `code/frontend/`, so that the deployment pipeline would have something real to build and verify end-to-end, rather than wiring up CI/CD against nothing.

Verified locally before wiring up any automation:

```
npm install
npm run build
```

Both succeeded, confirming the scaffold was a valid starting point.

4 CI/CD Pipeline: Vercel Deployment

Added `.github/workflows/vercel-deploy.yml`: a GitHub Actions workflow that deploys `code/frontend` to Vercel using the official Vercel CLI flow (`vercel pull` → `vercel build` → `vercel deploy --prebuilt`), triggered on pushes to `main` (production) and on pull requests (preview), scoped to only run when `code/frontend/**` changes.

4.1 Secrets

The workflow needs three GitHub Actions secrets: `VERCEL_TOKEN`, `VERCEL_ORG_ID`, and `VERCEL_PROJECT_ID`. Noted that Vercel tokens are scoped to an account/team, not to an individual project – the actual per-project targeting comes from `VERCEL_PROJECT_ID`, so the same token can be reused for a backend project added later.

Confirmed via `gh secret list` that all three secrets were present on the repository before the first CI run.

4.2 Debugging: Duplicated Root Directory Path

Error encountered on first real deploy run:

```
Error: ENOENT: no such file or directory, open
'.../code/frontend/code/frontend/package.json'
```

Relevant context: the Vercel project's own *Root Directory* setting (configured when the project was imported on `vercel.com`) was `code/frontend`. The workflow *also* set `defaults.run.working-directory` to `code/frontend` before invoking the Vercel CLI.

Key observation: `vercel build` applies the pulled project's Root Directory setting *relative to the current working directory*. Since the workflow had already `cd`'d into `code/frontend`, Vercel appended the same subpath again, producing `code/frontend/code/frontend`.

Solution: removed the `working-directory: code/frontend` default from the workflow, so all Vercel CLI steps run from the repository root and the project's own Root Directory setting is the single source of truth:

```
- defaults:
- run:
- working-directory: code/frontend
```

Because the two mechanisms (shell working directory and CLI-applied project setting) are not aware of each other and are not idempotent when composed – only one of them should determine the subdirectory.

5 Outcome

After the fix, the workflow ran green end-to-end (install → pull → build → deploy) and the frontend went live at:

`https://skillgrid-frontend-coral.vercel.app`

6 Follow-ups

- `mkdocs` GitHub Pages workflow also failed on its first run and needs a separate look.
- Backend architecture and its own Vercel/CI setup not yet decided.